Università degli Studi di Milano
Corso di Metodi Computazionali della Fisica

Processi $gg \to gg$ e $gg \to ggg$ in QCD: generazione simbolica delle ampiezze e calcolo del modulo quadro a tree-level

Edoardo Iannucci

Sommario

In questa relazione si presentano i risultati ottenuti nell'ambito del progetto per il corso di Metodi Computazionali della Fisica. L'obiettivo del lavoro è stato lo studio di processi di QCD al tree-level, in particolare $gg \to gg$ e $gg \to ggg$, mediante l'utilizzo di *Mathematica*, un software che impiega tecniche simboliche per la trattazione e la semplificazione di espressioni matematiche complesse.
Nello specifico, sono stati sviluppati algoritmi per la generazione iterativa di diagrammi di Feynman con un numero arbitrario di particelle esterne, a partire da una teoria di campo assegnata.
Un aspetto cruciale dell'analisi ha riguardato l'implementazione di algoritmi generalizzabili allo studio di processi $2 \to n$, con particolare attenzione alla crescita del costo computazionale al variare della complessità. È stata inoltre valutata l'incidenza dei limiti hardware, in termini di memoria e tempi di esecuzione, e le risorse computazionali necessarie per affrontare problemi via via più onerosi.

1 Introduzione

Lo studio di processi di scattering in QCD rappresenta una sfida notevole, dovuta alla complessità intrinseca delle espressioni coinvolte, che spesso crescono rapidamente con il numero di gluoni esterni. In molti casi risulta impraticabile un'analisi manuale, rendendo necessario l'utilizzo di strumenti computazionali per affrontare il problema in modo efficiente e strutturato. In questa relazione si descrive l'impiego di *Mathematica* per automatizzare e generalizzare il calcolo di tali processi, sia per analizzarli da un punto di vista teorico che per studiarne il comportamento computazionale.
A tal fine, sono state costruite delle metodologie per la creazione iterativa di diagrammi di Feynman a n particelle esterne.
I principali obiettivi fisici affrontati per entrambi i processi includono la verifica simbolica dell'identità di Ward per i processi $gg \to gg$ e $gg \to ggg$, sfruttando la decomposizione delle ampiezze in strutture di colore, e il calcolo delle ampiezze modulo quadro non polarizzate, con l'obiettivo di sviluppare una strategia quanto più generale e automatizzabile possibile.
Per ciascun algoritmo è stata condotta un'analisi dettagliata dei costi computazionali, valutando l'occupazione di memoria, i tempi di esecuzione e le risorse hardware necessarie. L'intero studio è stato svolto su una macchina con 16 GB di RAM e 10 core fisici, sfruttando la parallelizzazione

offerta da *Mathematica* per velocizzare i passaggi più onerosi. È stato inoltre considerato come tali requisiti scalino al crescere della complessità del problema, al fine di stimare l'hardware necessario per estensioni più ambiziose dello studio.

2 Generazione dei diagrammi

In questa prima sezione viene descritto il procedimento di costruzione dei diagrammi di Feynman per un dato processo, diviso in due fasi principali: la generazione della funzione di correlazione connessa con n punti esterni e la successiva definizione di una teoria di campo, necessaria per attribuire un significato fisico alle funzioni di correlazione ottenute.

2.1 Generazione grafi

In questa sezione descriviamo il procedimento di costruzione della funzione di correlazione connessa con n punti esterni.
Un singolo grafo è rappresentato da una lista di coppie di numeri interi, dove ciascuna coppia identifica una linea (propagatore). I punti esterni sono indicizzati con interi positivi, mentre i punti interni (vertici) sono etichettati con interi negativi.
Il punto di partenza dell'algoritmo è un semplice vertice connesso a tre linee esterne, che chiamiamo `grafobase`. Questo oggetto è rappresentato in codice come segue e illustrato graficamente in figura 1.

```
grafobase = {{1, -1}, {2, -1}, {3, -1}}
```

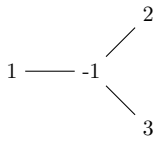


Figura 1: Vertice a tre linee esterne: punto di partenza del nostro algoritmo.

La generazione di tutti i grafi connessi per un processo con n punti esterni avviene in modo iterativo, a partire da `grafobase`. A ogni iterazione si creano nuovi grafi aggiungendo nuove linee esterne in tutti i modi compatibili con la struttura del grafo. Si assume che i vertici ammessi siano di tipo trivalente o quadrivalente, in accordo con la struttura delle interazioni puramente gluoniche considerate in questo studio.
Durante questo processo, i punti esterni già presenti rimangono invariati, mentre nuove linee vengono collegate progressivamente alla struttura esistente per raggiungere il numero desiderato di estremità.
In figura 2 sono mostrati tutti i grafi con quattro punti esterni ottenuti applicando un singolo passo dell'algoritmo a partire da `grafobase`.

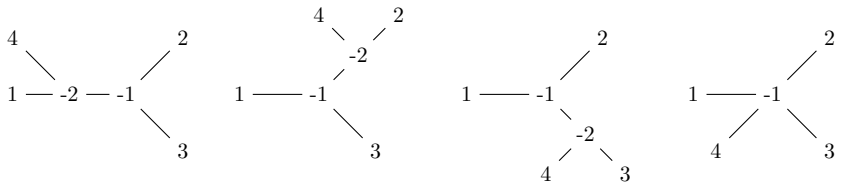


Figura 2: Grafi con quattro punti esterni.

Questo processo viene ripetuto sistematicamente ed esaustivamente, fino a ottenere tutti i grafi connessi aventi n linee esterne, rispettando le regole topologiche imposte dalla teoria di campo considerata.

È interessante osservare come il numero totale di grafi generati cresca rapidamente al crescere del numero di estremità. In tabella 1 sono riportati i valori ottenuti per diversi nn, mentre la figura 3 mostra chiaramente che tale crescita segue un andamento esponenziale.

n	Numero di grafi
4	4
5	25
6	220
7	2485
8	34300
9	559405

Tabella 1: Numero di grafi all'aumentare del numero di linee esterne.

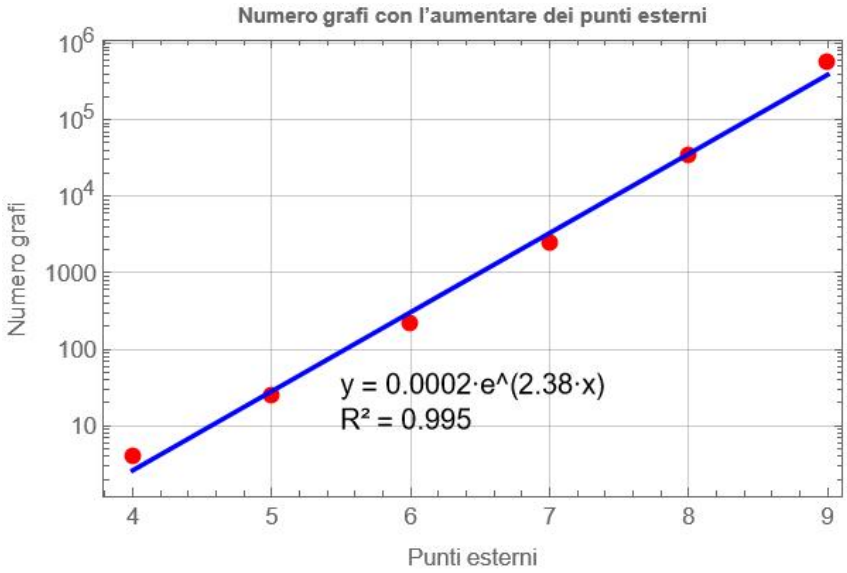


Figura 3: Crescita esponenziale del numero di grafi al variare del numero di linee esterne.

L'incremento è tipico dei problemi combinatori, dove anche un piccolo aumento del numero di elementi comporta un numero esplosivo di configurazioni possibili. Il buon accordo con il fit esponenziale è confermato dal coefficiente di determinazione R^2, calcolato da *Mathematica*, che quantifica quanto bene il modello esponenziale approssima i dati osservati (valore prossimo a 1 indica un'eccellente corrispondenza).

2.2 Dai grafi all'ampiezza

Dopo aver generato l'insieme completo di grafi, il passo successivo consiste nell'attribuire loro un contenuto fisico, traducendo la struttura combinatorica di ciascun grafo in contributi concreti all'ampiezza, dati dal prodotto di propagatori, vertici e stati esterni. Questo processo avviene attraverso i seguenti passaggi:

- **Attribuzione degli impulsi**: si assegna un quadrimpulso a ogni linea del grafo, assumendo convenzionalmente che tutti gli impulsi siano entranti nei vertici, imponendone la conservazione ad ogni vertice e a livello globale.
- **Implementazione della teoria e regole di Feynman**: a ciascuna linea interna viene associata una funzione propagatore `Prop`, mentre a ogni vertice si assegna una funzione di interazione, `V3` o `V4`, a seconda della valenza. Tali funzioni vengono poi esplicitate in termini applicando le regole di Feynman della teoria di riferimento, implementate manualmente nel nostro codice.

2.3 Manipolazione simbolica

Le ampiezze generate coinvolgono strutture tensoriali contenenti indici di Lorentz e di colore. Per trattarle in modo sistematico si sono implementate una serie di operazioni di manipolazione simbolica, attraverso l'utilizzo estensivo del comando `ReplaceRepeated` (`//.`), che consente l'applicazione iterativa di regole di sostituzione fino al raggiungimento di una forma stabile. Alcune delle procedure implementate sono:

- `LorentzSP` e `LorentzSPinternal`: gestiscono la contrazione tra vettori di Lorentz, restituendo automaticamente il prodotto scalare tra quadrivettori; `LorentzSPinternal` si occupa in particolare delle contrazioni tra indici interni.
- `ExpandLorentz`: applica le proprietà di linearità e omogeneità ai vettori di Lorentz.
- `colourSP`: effettua prodotti scalari tra oggetti che obbediscono all'algebra di colore.

Alcune operazioni simboliche fondamentali sono già integrate in *Mathematica*, come `Expand`, che sviluppa algebricamente le espressioni, e `Simplify`, che ne riduce la complessità.
L'uso di queste procedure comporta un rapido aumento dell'occupazione di memoria e un dilatarsi dei tempi di valutazione, in particolare per espressioni molto grandi o complicate.
La tabella 2 mostra l'evoluzione del carico computazionale associato ai principali passaggi di manipolazione simbolica applicati a quattro diagrammi di Feynman per $n = 4, ..., 7$.

Prestazioni computazionali dei passaggi simbolici su singoli diagrammi per $n = 4, ..., 7$

	$n=4$	$n=5$	$n=6$	$n=7$
Memoria occupata prima di espandere [B]	7.26E3	1.22E4	1.79E4	2.41E4
Memoria occupata dopo [B]	1.59E5	3.41E6	8.08E7	2.13E9
Tempo di esecuzione [s]	0.0020	0.0157	0.3429	30.0304
Memoria occupata dopo aver contratto [B]	8.87E4	1.18E6	1.72E7	2.73E8
Tempo di esecuzione [s]	0.0062	0.1491	5.1098	419.2328
Memoria occupata dopo aver semplificato [B]	3.42E4	3.28E5	3.49E6	4.23E7
Tempo di esecuzione [s]	~ 0	0.2472	84.8136	990.1970

Tabella 2: Tempi di esecuzione e occupazione di memoria (in byte) nei tre passaggi simbolici principali (`ExpandLorentz + Expand`, `LorentzSPinternal`, `Simplify`) applicati a singoli diagrammi con $n = 4 \cdots 7$.

Si osserva una differenza nel comportamento dei tre passaggi simbolici in termini di scalabilità temporale: mentre i tempi di esecuzione di `Expand` crescono in modo relativamente contenuto con n, le operazioni di contrazione e soprattutto di semplificazione mostrano un'escalation molto più marcata. È possibile inoltre notare che la dimensione delle espressioni generate cresce approssimativamente di un ordine di grandezza a ogni incremento di n. Questa crescita, già

rilevante a livello del singolo diagramma, si combina all'aumento esponenziale del numero di grafi, comportando come vedremo una rapida saturazione delle risorse computazionali disponibili.

3 Identità di Ward

Dopo aver implementato le procedure per la generazione automatica dei diagrammi di Feynman, abbiamo condotto un primo test significativo per valutare la capacità di *Mathematica* di gestire simbolicamente espressioni complesse: la verifica dell'identità di Ward.
A tal fine, è stato sviluppato un codice che automatizza tale verifica per i processi a n particelle esterne. L'ampiezza totale viene scomposta in sottostrutture di colore indipendenti attraverso l'applicazione dell'identità di Jacobi. La decomposizione è stata riconducibile alla formulazione proposta in [1], nota come base DDM (Del Duca, Dixon e Maltoni), e integrata nel nostro framework simbolico.
L'algoritmo implementato segue i seguenti passaggi:

1. Generazione dell'ampiezza tramite le regole di Feynman definite.
2. Espansione simbolica con `ExpandLorentz` e `Expand`, seguita dalla contrazione degli indici interni tramite `LorentzSPinternal`, per semplificare la struttura tensoriale delle espressioni.
3. Decomposizione dei fattori di colore tramite identità di Jacobi, fino a ottenere una scrittura in termini di combinazioni lineari degli elementi della base DDM [1].
4. Contrazione con momenti esterni e polarizzazioni, imponendo la conservazione del quadrimpulso a ogni vertice e sull'intera ampiezza.
5. Riscrittura in termini delle variabili di Mandelstam indipendenti, semplificazione finale con `Simplify` e verifica diretta dell'identità di Ward per ciascuna sottoampiezza.

3.1 Scomposizione in fattori di colore

Come anticipato, la verifica dell'identità di Ward richiede la scomposizione dell'ampiezza in strutture di colore che la soddisfino separatamente, rendendo necessario identificare una base di componenti cromaticamente indipendenti.
Una generica ampiezza gluonica al tree-level con n linee esterne può essere espressa come:

$$\mathcal{A}_n^{\text{tree}} = \sum_i c_i A_{ni} \tag{1}$$

dove

$$c_\sigma = f_{x_1 a_{\sigma_1} a_{\sigma_2}} f_{x_2 x_1 a_{\sigma_3}} \cdots f_{x_{n-3} a_{\sigma_{n-1}} a_{\sigma_n}} \tag{2}$$

sono i fattori di colore costruiti come prodotti concatenati di costanti di struttura, mentre A_{ni} rappresentano le sottoampiezze associate a n particelle esterne.
Il simbolo σ denota una permutazione degli indici esterni da 1 a n, tenendo conto della simmetria dovuta alla possibilità di scambiare gli indici all'interno della prima e dell'ultima coppia di costanti di struttura, nonché di permutare le due coppie tra loro. In totale si ottengono quindi $n!/(2 \cdot 2 \cdot 2)$ termini distinti. Tuttavia, le strutture di colore così ottenute non sono linearmente indipendenti, essendo legate tra loro dalle identità di Jacobi.

Per isolare i contributi realmente indipendenti si adotta invece la cosiddetta base DDM, che elimina la ridondanza cromatica imponendo due indici fissati agli estremi. In questa rappresentazione, l'ampiezza assume la forma:

$$\mathcal{A}_n^{\text{tree}} = \sum_i c_i^{\text{DDM}} A_{ni}^{\text{DDM}} \tag{3}$$

$$c_\pi^{\text{DDM}} = f_{x_1 a_1 a_{\pi_2}} f_{x_2 x_1 a_{\pi_3}} \cdots f_{x_{n-3} a_{\pi_{n-1}} a_n} \tag{4}$$

dove la somma è estesa a tutte le permutazioni π degli indici centrali $2, \ldots, n-1$, con 1 e n mantenuti fissi alle estremità. Questa scelta riduce il numero totale di strutture di colore indipendenti a $(n-2)!$.
La tabella 3 mostra chiaramente come crescono, al variare del numero di gluoni esterni, sia il numero di strutture di colore totali (dipendenti), sia quello delle componenti indipendenti selezionate tramite la base DDM.

Crescita delle strutture di colore al variare del numero di gluoni esterni

n	Strutture dipendenti	Strutture indipendenti
4	3	2
5	15	6
6	90	24
7	630	120
8	5040	720
9	40320	5040

Tabella 3: Numero di strutture di colore dipendenti ($\sim (n!/8)$) e indipendenti ($\sim (n-2)!$) al variare del numero di gluoni esterni n.

La decomposizione DDM dei fattori di colore si basa su un passaggio cruciale: riscrivere ogni struttura della forma (2) come combinazione lineare di termini nella base indipendente (4), attraverso l'applicazione sistematica delle identità di Jacobi.
L'algoritmo genera tutte le riscritture possibili di ciascun fattore di colore tramite sostituzioni del tipo

```
f[e_, g_, a_] f[e_, b_, d_] :> f[e, g, d] f[e, b, a] - f[e, g, b] f[e, d, a]
f[g_, e_, a_] f[e_, b_, d_] :> f[g, e, d] f[e, b, a] - f[g, e, b] f[e, d, a],
```

filtrando a ogni iterazione solo le trasformazioni che aumentano il numero di termini, fino a esprimere interamente ogni contributo nella nuova base. I coefficienti ottenuti in corrispondenza di ciascun elemento indipendente costituiscono le sottoampiezze associate.
L'algoritmo è costruito in modo da essere generalizzabile a un numero arbitrario n di gluoni esterni, ma il costo computazionale cresce rapidamente: la riduzione richiede infatti di scorrere l'intera ampiezza per identificare tutti i fattori di colore presenti, raccogliendo i coefficienti corrispondenti per ottenere le sottoampiezze, operazione che diventa rapidamente onerosa all'aumentare del valore di n.

La tabella 4 riassume i principali passaggi coinvolti nella verifica dell'identità di Ward. Si osserva che l'aumento del numero di gluoni esterni comporta una rapida crescita della memoria occupata dall'ampiezza espansa e contratta: si passa da poche centinaia di kilobyte a oltre 18 megabyte per $2 \to 3$, con un incremento di quasi due ordini di grandezza. Questa crescita riflette sia l'aumento esponenziale del numero di diagrammi sia la complessità media delle singole

ampiezze, sebbene quest'ultima possa variare significativamente tra grafi a causa di possibili semplificazioni strutturali.
Questo aumento si riflette con una crescita dello stesso ordine di grandezza nel tempo necessario per l'estrazione delle sottoampiezze indipendenti tramite decomposizione DDM, che diventa il passaggio computazionalmente dominante. L'algoritmo, infatti, deve esplorare un ampio spazio di riscritture mediante identità di Jacobi e confrontare ogni combinazione generata con la base indipendente, un processo inefficiente in presenza di strutture complesse.

Costi di verifica dell'identità di Ward

	$2 \to 2$	$2 \to 3$
Tempo di espansione e contrazioni interne [s]	0.0709	1.0950
Memoria occupata dopo contrazioni interne [B]	274'000	18'150'928
Tempo estrazione sotto-ampiezze indipendenti [s]	0.1411	10.0509
Tempo contrazioni esterne [s]	0.0953	7.2837
Memoria prima della semplificazione finale [B]	24'520	6'855'784
Tempo semplificazione finale [s]	0.1105	1.9428
Tempo totale [s]	0.4927	20.4535

Tabella 4: Confronto tra i costi computazionali per la verifica dell'identità di Ward nei processi $gg \to gg$ e $gg \to ggg$.

Come mostrato nella tabella 2, per il processo $2 \to 4$ una singola ampiezza espansa occupa circa 80 MiB; considerando i 220 diagrammi contribuenti, la memoria richiesta per rappresentare l'intera ampiezza supera i 17 GiB, un calcolo che, con la procedura implementata, richiederebbe l'utilizzo di macchine dotate di maggiore memoria.

4 Ampiezza modulo quadro non polarizzata

Come per la verifica dell'identità di Ward, anche il calcolo dell'ampiezza non polarizzata viene eseguito a livello di sottoampiezze di colore. L'ampiezza modulo quadro può essere scritta come

$$\left|\mathcal{A}_n^{\text{tree}}\right|^2 = \sum_{ij} c_i c_j^\dagger A_i A_j^\dagger = \sum_{ij} \tilde{c}_{ij} \tilde{A}_{ij} \tag{5}$$

dove $\tilde{A}_{ij}$ rappresenta il prodotto tra le sottoampiezze corrispondenti, contratte sulle polarizzazioni esterne. Quest'ultima contrazione si basa sull'identità di somma sugli stati di polarizzazione:

$$\sum_{\text{pol}} \epsilon^\mu \epsilon^\nu = -g^{\mu\nu} + \frac{p^\mu n^\nu + p^\nu p^\mu}{p \cdot n} \tag{6}$$

L'obiettivo è ottenere un'espressione finale per l'ampiezza quadrata interamente in funzione degli invarianti di Mandelstam e della costante di accoppiamento, priva di dipendenza esplicita dal vettore ausiliario n^μ, la cui scelta è arbitraria e fissata dal gauge.
La procedura di calcolo adottata si articola nei seguenti passaggi:

1. Estrazione delle sottoampiezze di colore a partire dall'ampiezza completa.
2. Separazione dei fattori di colore dai rispettivi coefficienti (le sottoampiezze), in modo da trattarli separatamente.
3. Riscrittura dei fattori di colore nella base DDM, usando la rappresentazione dei generatori $f_{bac} = -iT^a_{bc}$:
$$c_\pi^{\texttt{DDM}} = (-i)^{n-2}(-1)^P (T^{\pi_2} \cdots T^{\pi_{n-1}})_{a_1 a_n} \tag{7}$$

dove P è il numero di permutazioni necessarie per riordinare gli indici. Il prodotto modulo quadro dei fattori di colore consente di riscrivere c_{ij} come tracce di generatori:

$$c_{\pi}^{\texttt{DDM}} c_{\pi'}^{\texttt{DDM}\dagger} = \mathrm{Tr}[T^{\pi_1} \cdots T^{\pi_{n-1}} T^{\pi'_{n-1}} \cdots T^{\pi'_1}] \tag{8}$$

4. Contrazione delle sottoampiezze sulle polarizzazioni esterne mediante le funzioni `Expand` e `LorentzSP`.

5. Calcolo del prodotto tra la sottoampiezza contratta e la sua coniugata, seguito dall'imposizione della conservazione del quadrimpulso, espansione e contrazione. Il risultato è una matrice triangolare superiore $\tilde{A}_{ij}$, in cui i termini fuori diagonale (interferenze) compaiono con un fattore 2. Il numero di termini da calcolare per ottenere l'ampiezza quadrata è quindi $n_{cf}(n_{cf}-1)/2$, dove n_{cf} è il numero di fattori di colore.

6. Ricomposizione dell'ampiezza modulo quadro sommando i contributi $\tilde{A}_{ij}\tilde{c}_{ij}$, semplificazione finale dell'espressione e riscrittura in funzione degli invarianti di Mandelstam indipendenti.

Per tutto il calcolo abbiamo utilizzato le sottoampiezze di colore dipendenti. La tabella 5 chiarisce questa scelta: le sottoampiezze indipendenti richiedono molta più memoria e tempi di calcolo superiori. In particolare, la fase più intensiva consiste nel calcolo del prodotto tra sottoampiezze contratte e non contratte. In questa fase, l'utilizzo della decomposizione dipendente si è dimostrato cruciale per mantenere il costo computazionale sotto controllo.

Confronto tra sottoampiezze indipendenti e dipendenti nella contrazione di singole coppie

	Indipendenti		**Dipendenti**	
	$n=4$	$n=5$	$n=4$	$n=5$
Tempo estrazione strutture di colore [s]	0.1207	12.6210	0.1306	11.376
Memoria occupata da una sottoampiezza [B]	1.07E5	7.43E5	5.40E4	7.79E5
Tempo contrazioni esterne $n!/8$ sottoampiezze [s]	1.5060	180.704	1.1466	157.652
Memoria di una sottoampiezza contratta [B]	1.54E6	1.33E8	8.26E5	2.16E7
Memoria prodotto espanso [B]	1.73E8	—	4.85E7	8.77E9

Tabella 5: Confronto tra tempi di esecuzione e occupazione di memoria per la contrazione di singole coppie di sottoampiezze (non contratta e contratta) nei processi $gg \to gg$ ($n=4$) e $gg \to ggg$ ($n=5$), utilizzando sia la decomposizione in strutture di colore indipendenti che dipendenti.

Per il caso $n=5$ con sottoampiezze indipendenti, la valutazione completa del prodotto espanso non è stata possibile: la dimensione dell'espressione risultante ha superato i limiti di memoria gestibili. Già l'espansione del prodotto tra un quarto della sottoampiezza contratta e un quarto di quella non contratta richiede oltre 12 GiB.
I tempi e le risorse richieste per il calcolo di ciascun termine sono riportati nella tabella seguente:

Contrazione di sottoampiezze complesse e semplificazione finale per $n=4,\ 5$

	$n=4$	$n=5$
Memoria prima di espandere il prodotto [B]	880'280	22'376'736
Memoria dopo aver espanso e contratto [B]	202'512	9'078'488
Tempo per espandere e contrarre [s]	0.4576	85.5074
Tempo di conservazione del momento e semplificazione [s]	0.4214	28.7000

Tabella 6: Misure di memoria e tempi per il calcolo di un singolo elemento A_{ij}, ottenuto come prodotto tra sottoampiezza contratta e coniugata, seguito da semplificazione.

I tempi totali di calcolo dell'ampiezza quadrata non polarizzata risultano pari a 7.08 s per il processo $gg \to gg$ e a circa 4.2 ore (15036 s) per il processo $gg \to ggg$. Tali risultati sono confrontabili con quanto riportato in letteratura su [2] e [3].

5 Conclusioni

L'analisi ha mostrato che la complessità dei calcoli in QCD cresce rapidamente con il numero di gluoni esterni, a causa dell'aumento combinatorio dei diagrammi, della dimensione media delle singole ampiezze e del numero di sottoampiezze di colore. Già per $n = 5$, la verifica simbolica dell'identità di Ward ha richiesto fino a 18 MiB per la sola espressione contratta. Un'estensione a $n = 6$ dove il numero di grafi è pari a 220 implicherebbe un'esplosione nella memoria occupata: si stima che l'ampiezza contratta espansa possa superare i 10 GB, rendendo necessario l'uso di risorse hardware e tecniche di parallelizzazione più robuste.

Per quanto riguarda l'ampiezza modulo quadro, l'utilizzo di sottoampiezze di colore dipendenti ha permesso di contenere significativamente il carico di memoria e i tempi di esecuzione. Tuttavia, la dimensione delle sottoampiezze scala rapidamente; una stima approssimativa per $n = 6$ può essere ottenuta dividendo la dimensione complessiva dell'ampiezza per il numero di sottoampiezze dipendenti:

$$\frac{220 \times 8.08 \times 10^7}{90} \text{ B} \approx 1.98 \times 10^8 \text{ B} \approx 0.184 \text{ GiB} \tag{9}$$

Una volta effettuata la contrazione sulle polarizzazioni esterne, che introduce un fattore moltiplicativo del tipo 3^6, la dimensione stimata di ciascuna sottoampiezza contratta diventa:

$$\frac{220 \times 8.08 \times 10^7}{90} \times 3^6 \approx 1.45 \times 10^{10} \text{ B} \approx 13.5 \text{ GiB} \tag{10}$$

Queste stime, seppur indicative, mostrano chiaramente la difficoltà di estendere in modo diretto i calcoli a $n = 6$. È importante notare che tali analisi non tengono conto di eventuali semplificazioni simboliche che potrebbero ridurre l'effettiva occupazione di memoria; tuttavia, offrono un ordine di grandezza realistico dei requisiti computazionali attesi.
In conclusione, l'estensione a $n = 6$ e oltre comporta un salto computazionale esponenziale, rendendo necessarie risorse hardware avanzate e strategie di semplificazione sempre più efficaci per mantenere il problema gestibile.

Riferimenti bibliografici

[1] Vittorio Del Duca, Alberto Frizzo e Fabio Maltoni. "Factorization of tree QCD amplitudes in the high-energy limit and in the collinear limit". In: *Nuclear Physics B* 568.12 (feb. 2000), pp. 211–262. ISSN: 0550-3213. DOI: 10.1016/s0550-3213(99)00657-4. URL: http://dx.doi.org/10.1016/S0550-3213(99)00657-4.

[2] R. K. Ellis, W. J. Stirling e B. R. Webber. *QCD and Collider Physics*. Cambridge Monographs on Particle Physics, Nuclear Physics and Cosmology. Cambridge University Press, 1996.

[3] F.A. Berends et al. "Single Bremsstrahlung Processes in Gauge Theories". In: *Physics Letters B* 103.2 (1981), pp. 124–128. DOI: 10.1016/0370-2693(81)90685-7.